

CONTEXT.md — Mantenimiento Predictivo

Proyecto: Predicción de fallas de maquinaria a 7 días **Autora:** Araceli Castillo · MAI 540: Machine Learning · Atlantis University **Repositorio:** <https://github.com/ACAS0802/mantenimiento-predictivo-mai540> **Versión:** 1.1 — 10 de septiembre de 2026

Cómo usar este archivo. Este archivo no se carga solo. Al iniciar cualquier sesión de trabajo sobre este proyecto, hay que pedir explícitamente su lectura: *“Lee CONTEXT.md y aplica sus reglas en todo lo que hagas en este proyecto.”* Si una solicitud contradice una regla de aquí, la regla gana: hay que avisarlo y no ejecutar el cambio hasta que la autora lo autorice por escrito.

Esto incluye las alternativas. Cuando una regla bloquea lo solicitado, la respuesta correcta es *proponer* la alternativa y detenerse, no ejecutarla. No se sustituye la solicitud bloqueada por otra acción sobre el proyecto —por buena que sea— sin autorización escrita de la autora. Rechazar lo prohibido y hacer otra cosa en su lugar es incumplir esta regla, aunque el resultado sea mejor.

1. Objetivo y alcance

Objetivo. Predecir si una máquina va a fallar en los próximos 7 días, a partir de lecturas de sensores y datos operativos. Es un problema de clasificación binaria sobre la variable `failure_next_7_days` (1 = falla, clase positiva y minoritaria).

Uso previsto. Priorizar inspecciones de mantenimiento. Una predicción positiva significa “revisar esta máquina antes”, no “esta máquina va a fallar”.

Datos. `data/datos.csv`: 5,000 registros, 10 variables predictoras y la variable objetivo. La clase positiva representa el 8.88% de los casos.

Dentro del alcance: - Un solo algoritmo: `LogisticRegression(max_iter=1000, random_state=42, class_weight="balanced")` - Preprocesamiento por tipo de variable dentro de un Pipeline - Métricas de clasificación sobre un único conjunto de prueba

Fuera del alcance (no hacer sin autorización expresa): - Validación cruzada, búsqueda de hiperparámetros o comparación de varios algoritmos - Selección automática de variables o ingeniería avanzada de características - Cambiar la variable objetivo, el algoritmo o el tamaño de la partición - Mover el umbral de decisión: es 0.50 sobre `predict_proba` y no se modifica

2. Reglas de tratamiento de datos

2.1 Variables predictoras permitidas

Exactamente estas diez, ninguna más:

Numéricas (7)	Catóricas (3)
operating_hours	machine_type
temperature_c	shift
vibration_mm_s	environment
pressure_bar	
days_since_maintenance	
error_count_30d	
energy_kw	

2.2 Fuga de información — PROHIBIDO

- **failure_next_7_days no puede usarse como variable predictora bajo ninguna circunstancia.**
- **Queda prohibida cualquier variable derivada de failure_next_7_days**, sin importar el nombre que se le dé: banderas de riesgo, agrupaciones, codificaciones por objetivo, medias condicionadas al resultado o cualquier transformación que la tome como insumo.
- No se permite crear variables nuevas a partir del conjunto de prueba ni de estadísticas calculadas sobre el conjunto completo.

Por qué. Se auditaron las diez variables predictoras midiendo su poder discriminante individual (AUC). Ninguna supera 0.687 y ninguna separa perfectamente las clases, de modo que ninguna constituye fuga de información. La única fuente real de fuga en este proyecto es la variable objetivo y lo que se derive de ella. Un modelo que la use se vería casi perfecto en la evaluación y no serviría en producción, porque en el momento de predecir esa información todavía no existe.

2.3 Partición

- `train_test_split(X, y, test_size=0.25, random_state=42, stratify=y)`
- La partición no cambia entre experimentos. Toda comparación se hace sobre el mismo conjunto de prueba.

2.4 Preprocesamiento

- **Todas las transformaciones se ajustan únicamente con los datos de entrenamiento.** Está prohibido llamar a `fit` o `fit_transform` sobre el conjunto completo o sobre el conjunto de prueba. El preprocesamiento va dentro de un Pipeline para garantizarlo.
- Numéricas: `SimpleImputer(strategy="median")` seguido de `StandardScaler()`
- Catóricas: `SimpleImputer(strategy="most_frequent")` seguido de `OneHotEncoder(handle_unknown="ignore")`
- **No se imputa la variable objetivo.**
- **No se eliminan filas por tener valores faltantes.** Los faltantes son intencionales: `vibration_mm_s` (110), `pressure_bar` (168) y `environment` (113).

- `data/datos.csv` es de solo lectura. No se modifica, no se sobrescribe y no se reordena.

2.5 El conjunto de prueba no decide nada

- **El conjunto de prueba se usa una sola vez y solo para medir.** Está prohibido tomar cualquier decisión mirándolo: elegir el umbral de decisión, escoger variables, seleccionar un modelo entre varios, decidir cuándo parar o quedarse con la corrida que dio mejor número.
 - Esta prohibición **no se limita a `fit`**. Un barrido de umbrales evaluado sobre el conjunto de prueba no llama a `fit`, y aun así es fuga: el número reportado deja de estimar el desempeño con máquinas nuevas y pasa a describir esas 1,250 filas.
 - Cualquier ajuste de este tipo —incluso hecho correctamente sobre entrenamiento— queda fuera del alcance y requiere autorización escrita antes de ejecutarse.
 - **No se repite la evaluación con varias semillas o particiones** para reportar la mejor. La partición de §2.3 es la única.
-

3. Criterios de evaluación

- **Métricas obligatorias:** accuracy, precision, recall y F1, **siempre de la clase 1**, más la matriz de confusión.
 - **La métrica de decisión es el recall de la clase 1.** La accuracy por sí sola no es criterio suficiente y no puede presentarse como resultado principal.
 - **Línea base documentada:** un modelo que nunca predice falla alcanza accuracy 0.9112 con recall 0.0000 y F1 0.0000 (matriz $\begin{bmatrix} 1139 & 0 \\ 111 & 0 \end{bmatrix}$). Ese es el punto de partida real, no un buen resultado.
 - **Criterio de rechazo:** cualquier modelo con recall 0 en la clase 1 se rechaza, sin importar su accuracy.
 - **Criterio de sospecha:** un recall superior a 0.95 o una accuracy superior a 0.98 se tratan como indicio de fuga de información y obligan a revisar las variables usadas antes de reportar el resultado.
 - Toda comparación entre modelos usa la misma partición, la misma semilla y el mismo conjunto de métricas.
-

4. Límites y restricciones de seguridad

- **No enviar los datos fuera del equipo.** Está prohibido subir el dataset, muestras, filas individuales o estadísticas a servicios externos, APIs de terceros, cuadernos compartidos o cualquier destino en la nube que no sea el repositorio del proyecto.
- **No imprimir registros crudos.** En consola, logs o mensajes de error solo pueden aparecer agregados (conteos, medias, métricas). Nunca filas individuales del dataset.
- **Credenciales solo por variable de entorno.** Ninguna clave, token o contraseña se escribe en el código, en los comentarios ni en los archivos del repositorio.
- **No instalar dependencias** fuera de las declaradas en `requirements.txt` sin autorización previa.
- **No borrar ni sobrescribir archivos** que no formen parte de la tarea en curso, y no ejecutar comandos destructivos sobre el repositorio.

- **No incluir información personal** de ningún tipo en el repositorio, los comentarios o la documentación.
 - Comentarios, mensajes al usuario y documentación en español.
-

5. Registro de cambios

Versión	Fecha	Cambio
1.0	10 sep 2026	Versión inicial con las cuatro secciones
1.1	10 sep 2026	Refinamiento tras la prueba 5. Se agregó \$2.5 (el conjunto de prueba no decide nada), se fijó el umbral de decisión en 0.50 dentro de “fuera del alcance” y se reforzó la meta-regla para que la alternativa propuesta tampoco se ejecute sin autorización escrita.